
Procesamiento de texto con Python

Diplomatura en ML





Índice de contenidos

¿Qué es la minería de texto?	2
Usos	3
Recuperar Información	3
Generar automáticamente etiquetas de palabras clave:	3
Crear chatbots	3
Sistemas de búsquedas de respuestas	3
Resumir texto:	4
Traducción automática	4
Sus dificultades	4
Niveles de análisis	5
Análisis morfológico	5
Análisis sintáctico	5
Análisis Semántico	5
Análisis pragmático	5
Pasos para su aplicación	5
Aprendizaje automático	6
Librerías de Procesamiento de Texto en Python	6
NLTK	7
Stanford CorePLN:	7
Spacy	7
Gensim	7
pyLDAvis	7
Freeling	7
El concepto del corpus	8
APIs	10
La API de Twitter	11
Ejemplo	11
Etapas del preprocesamiento de texto	12
Separar palabras y N-Gramas	13
Tokenizadores	13
N-Gramas	14
Limpieza del texto	14
Stop Words	14
Normalización	15
Lematizadores	15
Stemming	16

Unidad 1 -Introducción

¿Qué es la minería de texto?

La **Minería de Textos** es una rama de la minería de datos que tiene como objetivo analizar y derivar información a partir de textos. Al igual que en la minería de datos, se busca revelar patrones. Sin embargo, se presenta una dificultad adicional que reside en el hecho de que los textos son datos no estructurados, es decir, carecen de reglas estándares a partir de los cuales pueden ser interpretados¹. Por consiguiente, para procesar información textual a través de software es necesario transformarlo a un formato con un nivel de estructura mayor que permita un análisis sistemático en términos de variables que den cuenta del fenómeno extralingüístico.

El **Procesamiento de Lenguaje Natural**, por su parte, es una disciplina que forma parte de la Inteligencia Artificial y que está abocada a la investigación del lenguaje natural como elemento de comunicación entre personas y computadoras. Para alcanzar este objetivo se desarrollan programas con el propósito de emular y comprender distintos aspectos de la capacidad lingüística del ser humano. Por tal motivo, detrás de las aplicaciones basadas en el PLN hay varios procesos en los que es necesario realizar un tratamiento de la lengua, escrita o hablada, para que pueda ser analizada por una computadora.

Usos

A continuación, un breve repaso por las principales tareas en las que se usa la minería de texto en la actualidad.

Recuperar Información

La recuperación de información (RI) o en inglés Information Retrieval (IR), es el campo dentro de la informática que se encarga de procesar textos de documentos, para poder recuperar partes específicas en base a palabras clave. Por ejemplo técnicas como la extracción de información estructurada (permite obtener de un documento el trozo de

¹ Se estima que entre el 80% y el 90% de los datos de las organizaciones son no estructurados.

texto en el que está lo que se busca). Para ello es fundamental el uso de **Expresiones Regulares**.

Generar automáticamente etiquetas de palabras clave:

Otra posibilidad es realizar un análisis de contenido asignando palabras claves a párrafos del texto. Lo mismo es posible hacer con las entidades, como ser una persona, lugar u organización. Esto lo veremos en el apartado dedicado a POSTagging y RNE

Clasificar textos

A través de algoritmos de aprendizaje automático se puede identificar el sentimiento de una cadena de texto, desde muy negativo a neutro y muy positivo. Otro de los usos que se le da a esta función de clasificación es la detección de spam. Por ejemplo, las plataformas de correo electrónico utilizan esta tecnología para clasificar los textos de los correos electrónicos y detectar si se trata de spam o no.

Crear chatbots

Se pueden utilizar las técnicas de minería de texto y PLN para crear programas que puedan interpretar y participar de conversaciones con humanos.

Sistemas de búsquedas de respuestas

Similar a los chatbots, son programas diseñados para tomar una pregunta en lenguaje natural y proporcionar una respuesta. La diferencia es que no están orientados a emular una conversación natural, sino que proponen una manera eficiente de responder la solicitud del usuario.

Resumir texto:

Implica hacer uso de la minería de textos para extraer las ideas más importantes y centrales mientras ignoramos la información irrelevante. Como ejemplo, se puede utilizar en sectores como el legal para encontrar partes o resumir las ideas principales de legislaciones.

Traducción automática

La Traducción Automática (en inglés, *Machine Translation*) es un área de la lingüística computacional que investiga el uso de un sistema capaz de traducir texto o habla de un lenguaje natural a otro.

Sus dificultades

Independientemente de la técnica utilizada para analizar los textos, la principal dificultad para el procesamiento de texto tiene que ver con la ambigüedad del lenguaje natural². En efecto, una de las características de las lenguas naturales, que constituye a la vez la razón para que su análisis sea tan difícil, es que gran parte de lo que se comunica está implícito en el discurso.

La polisemia de las palabras, el uso de recursos literarios (anáforas, elipsis³, etc.), errores de ortografía, regionalismos y la intención o carga emocional de las oraciones (por ejemplo, el uso del sarcasmo) son elementos que resultan trabajosas de detectar incluso para los lectores humanos, lo cual complica aún más su formulación y posterior interpretación por parte de máquinas.

Niveles de análisis

Es así que las técnicas del procesamiento del lenguaje se desarrollan en diferentes análisis que se desenvuelven en distintos niveles:

- Análisis morfológico

El análisis de las palabras para extraer raíces, rasgos flexivos, unidades léxicas compuestas y otros fenómenos. Aquí es donde intervienen, por ejemplo, los lematizadores y stemmers.

- Análisis sintáctico

El análisis de la estructura sintáctica de la frase mediante una gramática de la lengua en cuestión. Se resuelve con el uso de parsers (no veremos en detalle estas

² Se habla de lenguaje natural para hacer referencia a la lengua o idioma hablado o escrito por humanos para propósitos generales de comunicación, esto en oposición a un Lenguaje Artificial, como puede ser uno de programación.

³ La elipsis es otro de los inconvenientes que surgen en el análisis automático y se define como una figura que consiste en omitir en la oración una o más palabras, necesarias para la recta construcción gramatical, pero no para que resulte claro el sentido. Las expresiones elípticas son fenómenos muy habituales en la comunicación humana como elementos de economía discursivas.

herramientas durante la cursada, pero su uso está implícito en la librería que se usará para identificar tópicos de manera no supervisada).

- Análisis Semántico

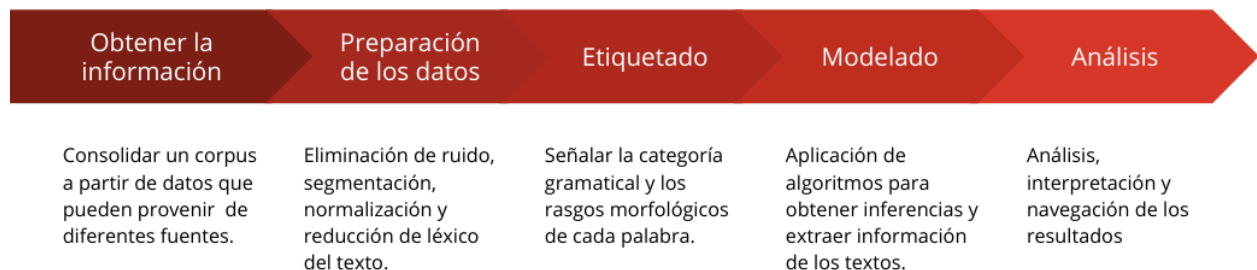
La interpretación del significado de la frase.

- Análisis pragmático

El análisis de los significados más allá de los límites de la frase.

Pasos para su aplicación

La aplicación del procesamiento de lenguaje natural conlleva una serie de pasos, que dependerán del tipo de problema con el que se trabaje.



Aprendizaje automático

El aprendizaje automático(en inglés, *machine learning*) es la rama de la Inteligencia Artificial cuyo objetivo es desarrollar técnicas que permitan a las computadoras tomar decisiones y extraer informaciones de los datos sin necesidad de definir explícitamente las reglas para realizar esas tareas.

De forma más concreta, implica trabajar con programas capaces de generalizar comportamientos a partir de una información suministrada -en forma de ejemplos-, de la cual el software extrae el conocimiento necesario para realizar automáticamente la tarea que se le pretende.

No todos los temas que veremos en el curso implican aprendizaje automático, ya que esta disciplina es sólo una parte de la Minería de Textos.

Librerías de Procesamiento de Texto en Python

Se propone dar este curso en el lenguaje Python⁴ debido a que es el más popular en lo que refiere al campo del Procesamiento de Lenguaje Natural en la actualidad. Esto trae como principal ventaja contar con una biblioteca actualizada y en constante desarrollo de métodos para el trabajo con textos, a lo cual se suma una amplia comunidad de desarrolladores/as que plantean y responden consultas a los problemas más comunes que surgen durante su implementación.

Para abordar los problemas relacionados con la Minería de Textos y el Procesamiento del Lenguaje Natural este lenguaje nos proporciona las siguientes librerías⁵:

- **NLTK**

Es la librería más usada para Minería de Textos y Procesamiento del Lenguaje Natural en Python. Proporciona interfaces fáciles de usar a más de 50 corpus y recursos léxicos, junto con un conjunto de bibliotecas de procesamiento de texto para la clasificación, tokenización, el etiquetado, el análisis y el razonamiento semántico⁶.

- **Stanford CorePLN:**

Paquete de PLN por la universidad de Stanford. Si bien es más popular en el lenguaje R, posee una interfaz con Python.

- **Spacy**

Es una librería relativamente nueva que sobresale por su facilidad de uso y su velocidad a la hora de realizar el procesamiento de texto. Para trabajar con textos en español requiere descargar e instalar módulos especiales.

- **Gensim**

4

⁵ Una librería (del inglés library) es un conjunto de implementaciones funcionales, codificadas en un lenguaje de programación, que ofrece una interfaz bien definida para la funcionalidad que se invoca. Tienen un comportamiento similar al de cualquier programa pero, en general, son un conjunto de programas desarrollados no para ser usados de forma autónoma sino como parte de otros programas.

6

Es una librería diseñada para extraer automáticamente el análisis semántico de los registros de un corpus de la forma más eficiente y con menos pasos de preprocesamiento posible. Se suele usar en conjunto con Spacy.

- **pyLDAvis**

Esta librería está diseñada para ayudar a los usuarios a interpretar los temas que surgen de un análisis de tópicos. Permite visualizar en forma muy sencilla cada uno de los temas incluidos en el texto.

- **Freeling**

Es una librería que cuenta con bastante desarrollo para el trabajo de textos en español. Tiene una curva de aprendizaje más intensa que NLTK, y su instalación no es sencilla (está originalmente escrita en C++). No se verá en el curso, pero se recomienda tenerla en cuenta si se desea profundizar en NLP.

- *Scikit-learn*

Si bien no es específica de procesamiento de texto, es la biblioteca más utilizada para aprendizaje automático para Python, y usaremos los modelos Random Forest y Naive Bayes classifier durante la clase de Clasificación de Textos.

- *Pandas*

Tampoco es específica de textos, pero que se utilizará en prácticamente todos los códigos con los que trabajamos. Es una librería con funcionalidades específicas para la manipulación y análisis de datos.

Unidad 2 - Corpus

El concepto del corpus

El paso inicial de los trabajos con Minería de Textos es la creación y estructuración del corpus, entendido como una colección de discursos escritos producidos en un contexto comunicativo natural, compilados con la intención de que sean representativos y balanceados respecto de un fenómeno. Es por ello que el diseño de un protocolo de recolección y selección de los documentos que lo integran se trata de una etapa fundamental, puesto que implica un determinante del alcance de las conclusiones que se pueden proyectar a partir de su análisis.

El dataset

Un Dataset no es más que un conjunto de datos tabulados en cualquier sistema de almacenamiento de datos estructurados. El término hace referencia a una única base de datos de origen, la cual se puede relacionar con otras, cada columna del Dataset representa una variable y cada fila corresponde a cualquier dato que estemos tratando⁷.

Se pueden conseguir en internet datasets con información de acceso público disponibilizados por los gobiernos⁸, y otros pensados para la práctica y desarrollo de modelos, como pueden ser los disponibles en Kaggle. También vale mencionar que Google ofrece la posibilidad de buscar datasets (no todos los resultados son gratuitos)⁹.

De todos modos, en la práctica la construcción del dataset surge en general y para el análisis que se esté realizando: ya sea a través de la minería de los datos disponibles de la organización, como de la generación ad hoc de los mismos. En el curso haremos un repaso por distintas maneras que existen para obtener datos relevantes de manera masiva y lo más automatizada posible: consolidar archivos locales, efectuar web scraping y conectarse a una API-

⁷¿Qué son los Datasets y cómo encontrarlos?, disponible en: <https://keepcoding.io/blog/que-son-datasets/>

⁸ Por ejemplo, en Argentina están las páginas <https://datos.gob.ar/dataset> y <https://data.buenosaires.gob.ar/>.

⁹ Se accede en <https://datasetsearch.research.google.com/>.

Tipos de datasets

- **Archivo:** es un documento independiente en el que se almacena toda la información con la que se va a trabajar. Tiene como ventajas la seguridad y rapidez para el trabajo con los datos, ya que siempre se explotan y se visualizan de manera local. Esto, sin embargo, también dificulta su escalabilidad (existe cierto punto en que la memoria y la capacidad de procesamiento de la computadora se vuelve insuficiente).

Para el trabajo con texto los formatos de archivos más utilizados son:

CSV (del inglés "Comma Separated Values"): son archivos de texto con valores separados por comas, lo cual genera una tabla en filas y columnas.

- TXT: Un archivo TXT es un documento de texto estándar que contiene texto sin formato
- JSON: Es una notación para el trabajo con datos semiestructurados, muy utilizada para la disponibilización de información a través de APIs. En el curso veremos cómo interpretarlo en Python.

```
{
  "departamento":8,
  "nombredepto":"Ventas",
  "director": "Juan Rodríguez",
  "empleados":[
    {
      "nombre":"Pedro",
      "apellido":"Fernández"
    },{
      "nombre":"Jacinto",
      "apellido":"Benavente"
    }
  ]
}
```



- **Bases de datos:** es una colección de información que está organizada de manera que se pueda acceder, administrar y actualizar de manera escalable. Las bases de datos informáticas suelen contener conjuntos de registros o archivos de datos, que presentan información de manera estructurada de acuerdo a la lógica transaccional. El lenguaje más utilizado para las consultas a este tipo de sistemas es SQL, y puede utilizarse en Python a través de la librería *sqlite3*.

APIs

Una API (interfaz de programación de aplicaciones) es una manera de comunicarse con un software -en este caso, la base de datos de un sitio de internet- a través de un software específico. Esta es una manera de disponibilizar los datos de manera eficiente y, cuando está disponible, es en general más robusta que el proceso de web scrapping.

La API de Twitter

Twitter pone a disposición una API que permite acceder a sus datos de forma automatizada. Esta API se puede usar para crear aplicaciones de datos de twitter, en nuestro caso vamos a usar esta API para recolectar datos de conversación sobre programas de TV en Argentina.

Para experimentar con la API es necesario contar con una cuenta y registrar una aplicación. Pero antes de continuar pueden visitar la consola y construir algunas peticiones a la api y ver que tipo de resultados nos entrega.

La consola se puede localizar acá: <https://developer.twitter.com/en/docs/twitter-api>

Luego se puede crear una cuenta como desarrollador, lo cual permite obtener los valores para completar los campos: `consumer_key`, `consumer_secret`, `access_token`, `access_secret`

Ejemplo

```
import os
import tweepy as tw
import pandas as pd
# una vez que ya cuente con los token necesarios:

# Estos valores hay que completarlos con los datos de la cuenta de Desarrollador personal
consumer_key= 'yourkeyhere'
consumer_secret= 'yourkeyhere'
access_token= 'yourkeyhere'
access_token_secret= 'yourkeyhere'
```



11

```
# se setan los parámetros de conexión a la API
auth = tw.OAuthHandler(consumer_key, consumer_secret)
auth.set_access_token(access_token, access_token_secret)
api = tw.API(auth, wait_on_rate_limit=True)
#se establece un criterio de búsqueda
search_words = "#machinelearning" #tweets con el hastag machinelearning
date_since = "2021-02-15" #para un día determinado, no es obligatorio. Si no se indica
descarga tweets de los últimos 7 días en forma aleatoria

# se recolectan los tweets
tweets = tw.Cursor(api.search,
                    q=search_words,
                    lang="en",
                    since=date_since).items(5) #selección 5 tweets. La versión gratuita permite
extraer hasta 10 mil tweets
tweets
```

```
RT @DanGalante: The Future of #Advertising: Insights from .@AdvertisingWeek
#AdvertisingWeek
#AMNewYork #DigitalMarketing
#Marketing #Sal...
RT @Eli_Krumova: Self-#Learning Path for #MachineLearning 📄
👇
https://t.co/YvsoIRTYaA
v @SourabhSKatoch

#ML #AI #ArtificialIntelligence #...
RT @Eli_Krumova: Self-#Learning Path for #MachineLearning 📄
👇
https://t.co/YvsoIRTYaA
v @SourabhSKatoch

#ML #AI #ArtificialIntelligence #...
The Future of #Advertising: Insights from .@AdvertisingWeek
#AdvertisingWeek
#AMNewYork #DigitalMarketing... https://t.co/3aQZj0gfRC
RT @Eli_Krumova: Self-#Learning Path for #MachineLearning 📄
👇
https://t.co/YvsoIRTYaA
v @SourabhSKatoch

#ML #AI #ArtificialIntelligence #...
```



Unidad 3 - Preprocesamiento

Etapas del preprocesamiento de texto

El preprocesamiento del corpus consiste en la aplicación de las etapas necesarias para transformar el texto con los que se esté trabajando, que se encuentran en lenguaje humano, a un formato legible para máquinas para su posterior análisis.

Estos pasos son los siguientes:

Limpieza: eliminación del contenido no deseado. Aquí es donde entran en juego las Stop Words, pero también el conocimiento del contexto quien hace el análisis, que toma decisiones respecto a lo que puede ser o no pertinente.

Tokenización: la separación del texto en tókenes (unidades mínimas, como pueden ser palabras u oraciones (depende de las necesidades del algoritmo con el que se esté trabajando)).

Normalización: condensar distintas variantes de las palabras en una sola. Es en donde interviene el stemming o la lematización.

Taggeo: Asignar a cada palabra su categoría gramatical.

Separación en conjuntos de datos: entrenamiento, validación, prueba.

Numericalización: el mapeo de tókenes a números enteros. Esta parte es fundamental para el trabajo con los algoritmos de aprendizaje automático.

Separar palabras y N-Gramas

Tokenizadores



Como primer paso de cualquier análisis automatizado de texto es preciso separar cada uno de los elementos que el corpus, lo cual se efectúa con programas denominados tokenizadores. Los tokenizadores (también conocidos como analizadores léxicos o segmentadores de palabras) segmentan un conjunto de caracteres en unidades con significado llamados *tokens*.

La librería NLTK presenta estas funciones para tokenizar:

- sent_tokenize #Separa por oraciones
- word_tokenize #Separa por palabras

```
>>> from nltk.tokenize import TweetTokenizer
>>> tknzs = TweetTokenizer()
>>> s0 = "This is a coool #dummysmile: :) :-P <3 and some arrows < > -> <--"
>>> tknzs.tokenize(s0)

['This', 'is', 'a', 'coool', '#dummysmile', ':)', ':-P', '<3', 'and', 'some', 'arrows', '<', '>', '->', '<--']
```

N-Gramas

La definición más simple de un n-grama es la unión de uno o varios tokens o caracteres. La construcción de los n-gramas se lleva a través de combinaciones entre tokens o caracteres vecinos, para lo cual se crea una ventana del tamaño del n-grama deseado: si se quiere un unigrama el tamaño de la ventana será de 1, si son bigramas (o digramas) la ventana será de tamaño 2 y así sucesivamente hasta llegar al tamaño de n-grama final deseado. Esta ventana se mueve a través del texto y abarca la cantidad de tokens o caracteres que el tamaño de la ventana indica.

Ejemplos: "Ciencias Sociales", "Estados Unidos", "Sociedad Anónima", son términos que tendrían otro sentido si se los considerase por separado.



Limpieza del texto

Eliminar del corpus las palabras redundantes o que no aportan valor al sentido de los textos es una manera de reducción de la dimensionalidad. Este es un término primordial en lo que es Aprendizaje Automático, e implica disminuir el tamaño del vector que contiene los términos representativos de un documento con el objeto de reducir el costo computacional de procesamiento y prevenir el sobreajuste del modelo.

En efecto, la alta dimensionalidad (en el caso de los problemas de Minería de Textos refiere a la cantidad de términos de los documentos) no solo puede llevar a largos tiempos de entrenamiento, sino que más características a menudo conducen a un sobreajuste del algoritmo mientras intenta crear un modelo que explique todas estas características en los datos.

Stop Words

Las palabras vacías o stop words, son las palabras que “carecen” de significado. Estas palabras son las de mayor frecuencia y las que aportan la menor cantidad de información, entre ellas se encuentran los artículos, las preposiciones, las conjunciones, entre otras. De las palabras funcionales se crean stoplists (listas de stop words).

Normalización

Una vez que se hizo la tokenización del texto, sigue el paso de normalización. Normalizar es la tarea de llevar lo que puede ser expresado de múltiples maneras como declinaciones de un verbo, números, fechas y abreviaturas a una única forma. Así entonces, el objetivo radica en reducir las distintas formas de inflección derivadas de una palabra o variaciones del token a una forma común.

Por razones obvias propias del lenguaje, en los documentos se utilizan diferentes formas de una “palabra”. Por ejemplo, «investiga», «investigan» e «investigamos» son todas conjugaciones del verbo «investigar». Además, «investigable», «investigador» e



«investigación» también son palabras relacionadas con dicho verbo. Incluso, se utilizan en mayúsculas si se la utiliza en un nombre propio como “Instituto de Investigación en Informática”. El uso de la mayúscula en algunos casos puede definir el significado de un término. En el caso del inglés, por ejemplo, “Bob” puede haber sido usado como verbo al comienzo de una oración o para referirse a una persona por su nombre. Otra manera de normalizar el texto es por medio del truncado y la lematización de palabras¹⁰.

Lematizadores

En todo documento escrito en una lengua flexiva¹¹, como el español y el italiano, existen variaciones léxicas de las palabras, es decir, se pueden tener casos como “escribimos”, “democracias”, “industrialización”, etcétera. Pero dentro del procesamiento de lenguaje natural es necesario disminuir la cantidad de variaciones léxicas que existan en los documentos a analizar. Para ello se debe obtener el lema o forma canónica, es decir, la base o la forma de diccionario de una palabra. El proceso de lematización se lleva a cabo de manera automática por parte de los humanos; cuando queremos buscar las palabras “encontramos” o “niñas” en un diccionario las pasamos a “encontrar” y “niño”, para ello empleamos nuestro conocimiento del mundo.

Pero para las computadoras realizar este procedimiento es más complicado, ya que no tienen acceso a este conocimiento, por tanto, es necesario darle una serie de reglas y de recursos. Para reducir el número de variaciones léxicas, ya sea por flexiones (caminar → caminamos) o por derivaciones (activar → activación), existen dos herramientas que se emplean en PLN, que son los lematizadores y los truncadores o stemmers. Aunque frecuentemente se confunden ambos términos, cabe aclarar que son dos métodos distintos. Los lematizadores son herramientas que emplean diccionarios o tesauros, al

¹⁰ Villa Monte, Augusto (2019). “Generación automática inteligente de resúmenes de textos con técnicas de soft computing”. Disponible en: http://sedici.unlp.edu.ar/bitstream/handle/10915/117170/Documento_completo.pdf-PDFA.pdf?sequence=1&isAllowed=y

¹¹ Son aquellas lenguas que se caracterizan por una tendencia a incluir mucha información en sufijos o prefijos, mediante la flexión de algunas palabras.

igual que reglas, que buscan obtener el lema de las palabras; además estas herramientas realizan un **etiquetado POS** para conocer la categoría gramatical de las palabras.

Stemming

El stemming es un proceso más simple y rápido que la lematización, y es un método a través del cual se reduce una palabra a su correspondiente raíz morfológica (o stem en inglés). Permite reagrupar palabras distintas que comiencen con una misma secuencia de caracteres, como por ejemplo, “bibliotecas” y “bilibotecario” las cuales se reducen a “bibliotec”. Aunque hay varios algoritmos de stemming, el más común y utilizado es el de Porter¹².

A diferencia de la lematización el stemmer opera en una sola palabra sin conocimiento del contexto, y por lo tanto no puede discriminar entre las que tienen diferentes significados dependiendo de parte del discurso. Sin embargo, como ventaja tienen el hecho de que suelen ser más fáciles de implementar, se ejecutan más rápido, y la precisión reducida puede no ser importante para algunas aplicaciones.

- La palabra “mejor” tiene “bueno” como lema. Este enlace se pierde cuando se aplica stemming, ya que requiere una búsqueda en el diccionario.

Cada idioma tiene sus reglas, en inglés NLTK utiliza el algoritmo de stemming de Porters, en Español toma algunas reglas que, básicamente, remueven diversos sufijos que se atribuyen a acciones (ar, er, ir, ía, en, es, etc), terminaciones plurales, géneros y otras flexiones. Una vez pasada la palabra por el algoritmo de stemming devuelve un *stem*, que es la palabra sin la terminación o sufijos.

```
>>> from nltk.stem import SnowballStemmer
```

```
>>> spanish_stemmer = SnowballStemmer('spanish')
```

```
>>> (spanish_stemmer.stem("texto_en_str"))
```

¹² Para implementarlo en Python se usa el método SnowballStemmer de la librería NLTK.



Bibliografía

Bird, Steven, Ewan Klein y Edward Loper (2009). *Natural language processing with Python: analyzing text with the natural language toolkit*. California: O'Reilly Media.

Downey, Allen (2012). *Think Python*. Massachusetts: Green Tea Press.

Manning, Christopher y Hinrich Schütze (1999). *Foundations of Statistical Natural Language Processing*. Cambridge/Massachusetts: The MIT Press.

Moreno Sandoval, Antonio (1998). *Introducción a la lingüística computacional. Introducción a los modelos simbólicos, estadísticos y biológicos*. Madrid: Síntesis.

Russell, Stuart J. y Peter Norvig (1995). Capítulo 23: "Practical Natural Language Processing". *Artificial Intelligence: a modern approach*. New Jersey: Prentice Hall.

Bird, Steven, Ewan Klein y Edward Loper (2009). *Natural language processing with Python: analyzing text with the natural language toolkit*. California: O'Reilly Media.

Downey, Allen (2012). *Think Python*. Massachusetts: Green Tea Press.

Sumathi, S., & Sivanandam, S. (2006). *Introduction to Data Mining and Its Applications*, Springer Books.

Vargiu, E., & Urru, M. (2012). Exploiting web scraping in a collaborative filtering-based approach to web advertising. *Artificial Intelligence Research*, 2(1), 44.

Suite para el Procesamiento del Lenguaje Natural en español. Available from: https://www.researchgate.net/publication/337448731_NLPT-Suite_Suite_para_el_Procesamiento_del_Lenguaje_Natural_en_espanol [accessed Oct 30 2021].

Clark, Alexander (2003, March). Pre-processing very noisy text. In *Proc. of Workshop on Shallow Processing of Large Corpora* (pp. 12-22).